

专业方向综合项目心得报告

项目名称：Air Rowing - 基于大模型的智能赛艇训练管理系统

小组成员：

学号	姓名	主要分工
2253721	陈茁杨	微服务架构 + 测试协调 Nacos 服务注册与发现： 管理所有微服务（用户服务、训练服务、AI 服务等）。 全局异常处理 & 日志监控（ELK）： 统一错误码设计，日志收集与分析。 组织 JMeter 压力测试（团队共同执行）。 编写 架构设计文档（微服务交互图）。 技术栈 Spring Cloud Alibaba Nacos ELK JMeter
2252843	杨宇琨	AI前端开发 + 部分后端联调 开发用户交互界面（Vue.js）： 实现 图片/视频上传 功能。 可视化展示 MediaPipe 关键点检测结果。 展示 大模型分析报告（标准度评分、改进建议）。 对接后端 API： 定义 前后端数据格式（JSON Schema）。 调用 MediaPipe 检测接口、大模型分析接口。 协助 Spring Cloud Gateway 配置（路由规则）。 参与 JMeter 压力测试。 技术栈 Vue3 Axios Three.js Spring Gateway
2254198	段子涛	MediaPipe 后端开发 + 存储与部署 开发 MediaPipe 姿势识别服务（Python/Flask）： 接收前端上传的图片/视频，调用 MediaPipe BlazePose 提取关键点。

		◦ 输出 归一化坐标 & 世界坐标（JSON 格式）。 • 优化检测性能： ◦ 调整模型参数，提升 实时性 & 准确率。 • 设计 非结构化数据存储（MinIO）： ◦ 存储用户上传的图片/视频。 • 负责 Docker 容器化部署： ◦ 编写 <code>Dockerfile</code>，部署 MediaPipe 服务。 技术栈 <code>Python</code> <code>MediaPipe</code> <code>Flask</code> <code>Docker</code> <code>MinIO</code>
2251528	王浩然	点集数据处理 + 大模型 API + 用户认证 • 点集数据处理： ◦ 将 MediaPipe 输出的关键点 转换为结构化 Prompt。 • 大模型 API 调用： ◦ 对接 DeepSeek 等 LLM，发送 Prompt 并解析返回结果。 ◦ 设计 姿态评估逻辑。 • 实现 Sa-Token 身份验证： ◦ 用户登录/注册接口（JWT 令牌）。 ◦ 权限控制（如教练 vs 普通用户）。 技术栈 <code>Spring Boot</code> <code>Sa-Token</code> <code>OpenFeign</code> <code>JSON</code> <code>数据处理</code>
2253209	刘佳朋	自研模型训练 + 模型服务化 + 限流熔断 • 训练 姿态分析模型（PyTorch/TensorFlow）： ◦ 输入：MediaPipe 关键点数据。 ◦ 输出：姿态标准度评分（0-100）。 • 模型优化： ◦ 对比 自研模型 vs 大模型 的效果差异。 • 封装模型为 REST API（Flask/Spring Boot）。 • 配置 Sentinel 限流规则（防止高并发崩溃）。 技术栈 <code>PyTorch</code> <code>Flask</code> <code>Sentinel</code> <code>模型评估</code>

1. 项目整体感受

1.1 团队协作体验

历经数月的协作与探索，我们的智能赛艇应用开发项目终于从构想走向落地。

项目的推进始终以例会制度为锚点，每周一次的线下讨论与日常线上同步成为常态。腾讯会议与飞书文档支撑了远程协作的骨架，需求文档的迭代、模型方案的讨论、代码冲突的解决，皆在一次次屏幕共享与文档批注中达成共识。初期因成员专业背景差异导致的沟通壁垒，在持续的“技术语言翻译”中逐渐消解。这种双向奔赴的磨合，让原本割裂的技术模块最终形成了有机的整体。

分工的动态调整贯穿始终。原定的模型训练计划因数据量不足陷入瓶颈时，团队曾面临责任分配的争议：究竟是全员投入数据采集，还是转向大模型适配？最终在导师的启发下，我们重新拆解任务优先级——一名成员专注系统其余部分的调整优化，另一组依托Mediapipe搭建基础框架，其余人员负责打通大语言模型的技术路线。这种“双线并行、阶段整合”的策略，既发挥了成员在算法优化与工程落地上的专长，也迫使我们在技术交叉点上反复对齐标准。

最深刻的教训来自一次接口联调失败：前端对姿态评分阈值的理解偏差，暴露出前期技术细节同步的疏漏。这促使我们建立了“核心指标白板推演”的惯例，用最朴素的方式确保认知同频。

回望这段历程，技术路径的修正远不及协作意识的进化更值得铭记。从初期对分工模糊的焦虑，到后期面对突发问题时的默契响应，那些深夜调试模型的轮岗值守、代码仓库的交叉审查、测试用例的集体评审，最终凝结成一份超越项目的收获——我们学会了如何将个体的专精转化为团队的合力，也在反复试错中触摸到了工程化协作的本质。项目终有终点，但这份在争议中寻找共识、在瓶颈中重构思路的经验，将成为未来技术实践中最坚实的底色。

1.2 项目完成度评价

与初期目标的对比（是否达成预期）

历经数月的努力，我们的智能赛艇应用开发项目在多个方面都达到了甚至超出了初期设定的目标。首先，在技术实现上，我们成功地将MediaPipe姿势识别技术与大语言模型相结合，实现了训练动作的实时数字化分析，并能够为用户提供专业的智能分析建议和标准度评分。此外，通过采用微服务架构，系统不仅具备了良好的扩展性和稳定性，同时也确保了高可用性数据存储方案。

在用户体验方面，本项目从用户上传图片或视频开始，到最终获得个性化的改进建议，整个流程流畅自然，极大地提升了用户的参与感和满意度。特别是3D姿态可视化界面的设计，使得运动员的动作评估更加直观准确，这一点得到了团队内部及初步用户测试中的广泛好评。

然而，在项目推进过程中也遇到了一些挑战，比如由于数据量不足导致原定自研模型训练计划陷入瓶颈，但团队迅速调整策略转向大模型适配，这一灵活应对措施保证了项目的顺利进行，并未对最终成果造成影响。

最具成就感的环节

1. 功能实现：最令人感到自豪的是实现了从MediaPipe关键点检测到大语言模型智能分析的全流程自动化处理。这一过程涉及到了多学科知识的应用和技术的深度融合，包括但不限于计算机视觉、人工智能算法以及人机交互设计等。特别是在优化了前后端数据传输格式后，解决了用户上传高帧率视频时前端页面卡顿的问题，这不仅是技术上的突破，也为后续类似问题提供了有效的解决方案模板。
2. 用户好评：当看到早期用户对我们产品给予积极反馈时，那种满足感难以言表。特别是运动员们表示，通过使用我们的系统，他们能更清晰地认识到自身存在的问题并得到针对性指导，这对于提高训练效率具有重要意义。此外，教练们也对系统提供的详细数据分析报告给予了高度评价，认为它有效地辅助了日常教学工作，提高了整体训练质量。

综上所述，尽管项目过程中遇到了若干预料之外的技术难题，但凭借团队成员之间的紧密合作与不懈努力，我们不仅圆满完成了预定目标，还在某些领域取得了超出预期的成绩。这些成就不仅体现在具体的软件功能实现上，更重要的是在整个团队协作能力和个人技能成长方面的显著提升。

1.3 个人感悟——对于整体项目

陈茁扬：

作为微服务架构的设计者与测试协调员，这次项目让我深刻体验到了从单体架构向微服务架构转型的复杂性与价值。记得项目初期，当各个模块都在单体应用中耦合开发时，一旦某个服务出现问题，整个系统都会受到影响。而当我成功搭建起基于Nacos的服务注册发现中心后，看到各个微服务能够自主注册、动态发现并实现负载均衡时，那种系统架构从混沌走向清晰的成就感难以言表。

最具挑战性的环节是建立全局异常处理与ELK日志监控体系。当时团队成员分布在不同的业务模块，异常信息散落在各个日志文件中，排查问题如同大海捞针。为了解决这个痛点，我设计了统一的错误码规范和日志收集策略，通过Elasticsearch、Logstash、Kibana构建起完整的监控链路，让分布式系统的运行状态一目了然。尤其是在压力测试阶段，当JMeter模拟高并发请求时，能够实时监控各个服务的性能指标和错误分布，这种掌控感让我真正理解了"可观测性"对于分布式系统的重要性。

团队协作中最温暖的时刻，是深夜调试微服务间通信问题时的相互支撑。当段子涛的MediaPipe服务偶尔出现超时，杨宇琨的前端页面随之卡顿时，我们三人在腾讯会议里一行代码一行代码地分析调用链路，从Nacos的服务健康检查到网关的路由配置，再到超时重试机制的参数调优。那种并肩作战解决技术难题的默契，让我感受到真正的团队精神不是分工明确后的各自为战，而是在关键时刻能够跨越边界、相互补位的信任与担当。

杨宇琨：

在这次团队协作中，我深刻体会到了技术攻关的挑战与团队协作的温暖。记得调试mediapipe前端展示时时，前端渲染的3D关键点总是无法完全在图片中完全展示，我和段子涛连着语音反复测试，从坐标系转换公式到着色器代码逐行排查，当最终看到完整的点集展示时，那种疲惫又兴奋的感觉至今难忘。还有和王浩然对接大模型接口时，我们为了把晦涩的专业术语转换成通俗建议，一起设计了术语转换词典。这些经历让我明白，真正的技术突破往往发生在跨团队的思维碰撞中，当负责后端同学主动帮我优化数据包体积，当段子涛耐心解释姿态评分逻辑时，我感受到的不仅是问题被解决，更是一

种被支持的踏实感。这次项目就像一场接力赛，每个人都在自己的赛段全力奔跑，又稳稳地把成果交到下个人手中，这种信任和默契，或许比技术本身更珍贵。

段子涛：

作为Mediapipe模块的负责人，这段经历让我深刻体会到技术落地与团队协作的双重挑战。起初我专注于模型本身的调优，却忽略了技术输出与团队需求的匹配——当杨宇琨反馈前端渲染效果不稳定时，我才发现自己默认的坐标系定义与前端展示逻辑存在偏差。这次教训促使我主动参与跨模块讨论，用更直观的示例数据替代晦涩的技术文档，让抽象的算法逻辑转化为全队可理解的通用语言。尤其在与大模型组对接时，我们通过场景化数据模拟提前验证交互流程：将姿态关键点转化为自然语言描述，再反向测试评分逻辑是否符合预期。这种以终为始的协作方式，不仅缩短了技术验证周期，更让我意识到，工程化落地的核心从来不是单点最优，而是全局协同。

王浩然：

在这次项目中，我深刻体会到了“技术落地”背后的复杂性与挑战。作为点集数据处理和大模型API对接的核心成员，我原本以为只要把关键点转换成自然语言描述、调通DeepSeek接口就完成了任务。但当杨宇琨提出前端展示需要更结构化的姿态评估维度时，我才意识到自己输出的数据格式并不能直接支持可视化渲染；而当刘佳朋反馈自研模型评分与大模型结果存在偏差时，我又被迫回过头重新审视评分逻辑的一致性。

这些经历让我明白，真正的技术实现不是孤立的模块完成度，而是要站在整个系统的角度去思考问题。于是，我开始主动参与跨组讨论，尝试用更通俗的语言解释姿态评估规则，并与段子涛一起设计了一套通用的数据样例库，用于验证不同模块之间的交互是否符合预期。那段日子我们常常在飞书文档里反复修改字段定义，甚至为了一个“动作偏移角度”的单位说明就开了三次线上会议。虽然过程繁琐，但当我看到最终用户报告中的建议既专业又易懂时，我知道这一切都是值得的。

最让我感动的是团队成员之间的相互信任与支持。记得有一次因为大模型返回内容格式不稳定，导致前端展示出现异常，杨宇琨没有责怪，而是第一时间帮我定位问题，还主动优化了前端的容错机制；而在测试评分一致性时，刘佳朋主动提出可以协助调整模型权重，帮助我更快地收敛评分逻辑。这些看似微小却温暖的协作瞬间，让我真正感受到：技术可以学习，但这种彼此理解、共同进退的团队精神，才是最宝贵的收获。

刘佳朋：

这次项目让我深刻体验到了跨学科合作的魅力和挑战。刚开始时，我们几个不同专业背景的成员在沟通上确实存在“专业壁垒”，计算机专业的同学更关注算法效率和代码实现，而体育专业的成员更注重动作的生理学合理性。但通过多次深入的交流讨论，我逐渐学会了用更通俗的语言解释技术概念，也开始理解运动科学的专业知识。这种思维碰撞不仅拓宽了我的知识视野，更重要的是让我认识到，真

正有价值的创新往往产生在学科交叉的边界上。当我们成功将深度学习技术与运动训练需求完美结合时，那种成就是单纯的技术实现无法比拟的。

2. 主要收获

2.1 专业能力提升

本次项目中，我们实践应用了课堂中学到的知识。

课程中学习的敏捷开发、需求分析等项目管理方法在团队协作中得到验证。例如，通过飞书文档构建共享任务看板，将需求拆解为算法、前端、后端模块，每日站会同步进度；Git分支管理策略（主分支+功能分支）确保代码整合稳定性，避免冲突。这种结构化流程使原本模糊的“软件开发生命周期”概念转化为可操作的实践。

在姿态评估模块开发中，依据软件测试课程中的黑盒测试思想，针对姿态打分逻辑设计边界值、异常值用例（如输入角度超出人体关节活动范围），并通过JUnit框架实现单元测试自动化；利用Postman对RESTful API进行压力测试，验证高并发场景下系统的鲁棒性。测试覆盖率从初期60%提升至85%。

此外，我们还学习了一些新技术与新工具的使用。

首次接触Mediapipe时，需从零理解其坐标系定义与关节点输出格式，通过编写Python脚本解析JSON数据并映射至本地姿态评估规则；大语言模型（LLM）集成阶段，学习Prompt Engineering技巧，将姿态参数结构化输入模型，并通过few-shot示例引导生成符合运动学原理的评语。这一过程深化了对模型微调与接口封装的认知。

后端采用Spring Boot框架重构原有单体架构，实践课程中学到的MVC模式与依赖注入机制，利用Spring Data JPA简化数据库交互。前端团队采用Vue.js框架实现动态交互，通过组件化开发模式拆分姿态可视化模块与用户控制面板，利用Axios封装HTTP请求完成与后端的数据交互。开发过程中，我们特别注重前后端联调规范，通过Swagger文档同步接口定义，采用Postman进行多场景测试，有效降低了协作成本。

2.2 软技能成长

2.2.1 团队沟通协调能力的提升

在本次项目中，团队成员之间的沟通与协调能力得到了显著提升。初期，由于对各自负责模块的理解存在差异，导致在联调过程中遇到了一些障碍。例如，在前端与后端的数据交互定义上，起初由于缺乏明确的接口文档和数据格式约定，造成了多次不必要的返工。为了改善这种情况，我们引入了Swagger工具来规范前后端接口文档，并通过每日站会分享各自的进度和遇到的问题，确保信息透明化、问题及时解决。

随着项目的推进，我们更加注重利用可视化工具来辅助沟通。比如，当讨论复杂的姿态评估算法时，我们会使用白板或在线绘图工具共同绘制流程图，帮助非技术人员更好地理解技术实现细节；对于用户界面的设计，则通过原型工具制作交互式Demo，以直观的形式展示设计思路，大大提高了团队内部以及与外部利益相关者之间的沟通效率。

此外，团队还建立了定期的技术分享会机制，鼓励成员分享自己在开发过程中学到的新知识或技巧，如如何优化Vue.js组件性能、Python脚本调试技巧等。这种方式不仅促进了知识共享，也增强了团队成员间的信任感和凝聚力。

2.2.2 时间管理与任务分解经验

时间管理和任务分解是本次项目成功的关键因素之一。在项目初期，我们就制定了详细的项目计划，将整个开发周期细分为多个阶段，并为每个阶段设定了明确的目标和里程碑。通过飞书文档构建的任务看板，我们将需求按照功能模块进行拆解，分配到具体的个人或小组，确保每个人都能清晰地了解自己的职责所在。

同时，我们也学会了合理设置缓冲期，考虑到开发过程中可能出现的不确定因素。例如，在对接MediaPipe服务与大语言模型的过程中，原本预计一周完成的工作因为数据格式不兼容等问题延长到了两周。幸好我们在项目规划时预留了一定的弹性时间，避免了因突发情况导致的整体进度延误。

在日常工作中，团队成员也逐渐养成了良好的时间管理习惯。每天早晨的站会不仅是同步进度的机会，也是重新审视当天工作任务优先级的时刻。通过这种方法，我们可以确保最重要的任务得到最先处理，并且能够灵活应对任何临时出现的紧急情况。

总的来说，这次项目不仅让我们在专业技能方面取得了长足进步，更重要的是提升了我们的软技能，包括但不限于团队沟通协调能力和时间管理能力。这些宝贵的经验将成为未来职业生涯中的重要财富。

2.3 个人收获

陈茁扬：

专业能力提升

这次项目极大地拓展了我在分布式系统设计和运维监控方面的技能边界。从零开始搭建微服务架构，让我深度掌握了Spring Cloud Alibaba全家桶的实际应用：通过Nacos实现服务注册发现和配置管理，用Gateway统一网关路由，结合Sentinel实现限流熔断。最具挑战性的是设计全局异常处理机制——我建立了一套分层的错误码体系，从底层的技术异常（如数据库连接失败）到业务异常（如用户权限不足），每类错误都有标准化的处理流程和用户提示。

ELK日志监控体系的搭建让我对可观测性有了全新认知。通过Logstash收集各微服务的日志，在Elasticsearch中建立索引，最终通过Kibana仪表盘实现可视化监控。当系统出现问题时，我能够通过日志关联分析快速定位问题所在的服务节点和代码位置，这种从"盲人摸象"到"一览众山"的技能跃迁，让我真正理解了分布式系统运维的核心要素。

软技能成长

作为测试协调员，我在这个项目中最大的成长是学会了如何在复杂项目中建立有效的沟通机制和质量管控流程。面对五个不同专业背景的团队成員，我逐渐摸索出一套"技术翻译"的方法：将抽象的架构概念转化为具体的业务场景，用流程图和时序图替代冗长的文字描述，让每个人都能清晰理解系统的整体运作机制。

更珍贵的收获是在处理突发问题时培养出的危机应对能力。记得在项目演示前一晚，微服务网关突然出现路由配置错误，导致前端无法正常调用后端接口。面对紧急情况，我迅速组织了线上紧急会议，与相关同学分工协作：一边回滚配置恢复服务，一边排查问题根因，同时准备备用方案。那种在压力中保持冷静、统筹全局的经历，让我深刻理解了技术leadership的内涵——不仅要有过硬的技术能力，更要具备在关键时刻凝聚团队、快速决策的综合素质。

杨宇琨：

专业能力提升

这次项目让我对AI与前端融合的开发模式有了更深的理解。从最初只能简单调用MediaPipe接口，到后来能够独立实现带点集渲染器，我不仅掌握了前端编程和性能优化技巧，更重要的是学会了如何将算法输出转化为直观的用户体验。记得为了优化关键点渲染的流畅度，我花了整整三天研究Web的多线程通信，最终将主线程的渲染耗时从120ms降到了40ms。而当大模型返回的分析结果出现延迟时，我又通过设计loading消息框提升了用户感知速度。这些实战经验让我明白，前端开发者的价值不仅在于界面实现，更在于成为连接AI黑盒与用户认知的桥梁。

软技能成长

如果说技术能力是硬实力，那么这次项目让我在团队协作和沟通上的成长则更为珍贵。起初，我对后端的数据格式和算法逻辑理解不够深入，导致联调时频繁出现误解。但通过主动参与接口设计讨论、学习使用Swagger文档协同工具，我逐渐能够用更专业的语言与团队成员对话。最难忘的是在项目中期，当进度因需求变更受阻时，我尝试用可视化原型代替文字描述，将抽象的技术方案转化为直观的流程图和交互Demo，意外地大幅提升了沟通效率。这种"用设计思维解决协作问题"的体验，让我意识到软技能不是虚无缥缈的概念，而是能实实在在推动项目前进的生产力。当看到最终产品上线时团队成员们发自内心的击掌庆祝，我忽然懂得：技术或许会过时，但这些在协作中培养出的理解力、同理心和解决问题的能力，才是工程师最持久的竞争力。

段子涛：

专业能力提升

这次项目让我对Mediapipe的实际应用有了深刻理解。最初接触时，我只是机械地调用官方接口，但随着需求细化才发现，单纯依赖默认参数会导致关键点检测结果不稳定——比如用户上传的图片中，赛艇运动员的某些动作角度稍偏，系统就会漏掉关键节点。为了提升鲁棒性，我尝试调整模型的置信度阈值，并通过大量样本测试找到平衡点。这个过程让我意识到，技术落地远不止“能跑起来”那么简单，更需要结合场景反复验证细节。

与杨宇琨对接时，我也学到了如何将抽象的技术指标转化为团队共识。初期我们因关键点坐标的定义方式产生分歧，后来干脆用白板画出坐标系，边讨论边模拟数据流转，最终统一了前后端的交互逻

辑。这种从“各自为战”到“协同验证”的转变，让我明白技术方案的落地必须建立在清晰的沟通基础上。

软技能成长

作为Mediapipe模块负责人，我一度陷入“闷头写代码”的误区，直到某次例会上听到杨宇琨提到前端渲染卡顿的问题，才意识到自己输出的原始坐标数据存在冗余维度。这次教训让我学会主动同步进展：每天在飞书文档更新关键点处理逻辑的改动，并用简单的示意图标注变化点。当大模型组提出需要补充关节角度描述时，我也提前预留了数据字段扩展空间，避免了后续大规模返工。

最难忘的是中期技术路径调整时的分工博弈。原本计划自研模型训练，但因数据量不足被迫转向大模型适配。面对任务重心的突然转移，我们快速重组资源：我和王浩然搭档，将Mediapipe的输出结果直接转化为大模型可解析的自然语言描述；刘佳朋则专注验证评分逻辑的一致性。这种灵活切换让我体会到，真正的团队协作不是按部就班执行分工，而是在变化中找到最优解。

王浩然：

专业能力提升

这次项目让我对点集数据的处理和大模型的应用有了更深入的理解。起初，我只是简单地将MediaPipe输出的关键点以JSON格式存储，但随着需求推进，我意识到原始数据存在冗余、噪声甚至缺失的问题。为了提升后续分析的准确性，我开始研究如何清洗、归一化这些关键点，并尝试将其转化为自然语言描述，以便更好地适配大模型输入要求。这一过程不仅让我掌握了结构化数据与自然语言之间的映射逻辑，也提升了我在文本生成、Prompt工程方面的实战能力。

在对接DeepSeek等LLM时，我还学习了如何设计合理的评分逻辑和改进建议模板，使得大模型输出的结果更具可解释性和实用性。比如通过引入姿态角度偏差、关节位置偏移等量化指标作为Prompt的一部分，显著提高了反馈内容的专业性和针对性。这种从“数据提取”到“信息转化”的思维转变，让我真正体会到AI系统中“数据即输入、语言即输出”的融合魅力。

软技能成长

在项目初期，我对跨模块协作的理解还比较片面，认为只要完成自己的任务即可。然而当杨宇琨提出前端展示需要更直观的姿态评估维度时，我才意识到自己设计的数据结构可能无法直接支持可视化渲染；而当刘佳朋反馈自研模型评分与大模型结果不一致时，我又被迫重新审视评分逻辑的一致性。这些经历促使我主动参与多组联调会议，学会站在不同角色视角思考问题。

最让我印象深刻的是与段子涛合作转换关键点为自然语言描述的过程。我们最初各自按照技术术语表达，导致沟通效率很低，后来干脆一起梳理出一套统一的“业务词汇表”，用大家都理解的语言定义动作标准、姿态偏差等概念，极大提升了协作顺畅度。这也让我明白，真正的软技能不是“会说话”，而是能够将复杂的技术问题翻译成团队成员都能理解的语言。

此外，在实现Sa-Token身份验证的过程中，我也学会了如何平衡安全性和用户体验。通过与杨宇琨反复推敲登录流程、权限控制策略，最终设计出一套既满足教练用户管理需求又不影响普通用户操作体验的身份体系。这种从“单点功能”向“整体体验”延伸的思维方式，是我在这次项目中最宝贵的收获之一。

刘佳朋：

专业能力提升

这次项目让我对点集数据的处理和大模型的应用有了更深入的理解。起初，我只是简单地将MediaPipe输出的关键点以JSON格式存储，但随着需求推进，我意识到原始数据存在冗余、噪声甚至缺失的问题。为了提升后续分析的准确性，我开始研究如何清洗、归一化这些关键点，并尝试将其转化为自然语言描述，以便更好地适配大模型输入要求。这一过程不仅让我掌握了结构化数据与自然语言之间的映射逻辑，也提升了我在文本生成、Prompt工程方面的实战能力。

软技能成长

这个项目最大的收获是让我从单纯的技术开发者转变为具有产品思维的工程师。以前我总是专注于模型的准确率和技术的先进性，但在与体育专业同学和实际用户接触后，我开始思考技术如何真正服务于用户需求。比如，我们最初设计的建议反馈过于技术化，运动员很难理解；后来通过用户测试和反馈，我们将复杂的技术指标转化为简单易懂的动作提示，这种转变让我深刻理解了"技术为人服务"的理念。现在回想起来，这种从技术导向到用户导向的思维转变，可能比掌握任何一项具体技术都更有价值，它将伴随我整个职业生涯。

3. 项目创新点

1. 技术创新

(1) 传统方案的改进

- **传统方式：**依赖教练现场指导或简单视频回放分析，缺乏客观量化标准
- **本项目创新：**
 - 采用**MediaPipe姿势识别技术**，实现训练动作的实时数字化分析
 - 结合**大语言模型**，将专业教练经验转化为智能分析建议
 - 开发**自主训练模型**，提供标准度评分和个性化改进方案

(2) 算法优化

- 创新性地将**BlazePose算法**与**GHUM技术**结合
- 提出**16个关键身体地标**的检测方案，精准捕捉赛艇运动特征

- 实现**归一化坐标**和**世界坐标**的双重分析体系

2. 形式创新

(1) 多学科融合

- 计算机技术**：微服务架构+AI算法
- 体育科学**：专业赛艇动作分析标准
- 人机交互**：直观的3D姿态可视化界面

(2) 展示方式的创意

动态分析报告：

- 实时生成训练姿势的3D模型
- 可视化标注动作偏差点
- 提供改进建议动画演示

智能交互：

- 语音实时指导功能
- 训练数据历史对比
- 个性化训练计划生成

3. 应用价值创新

- 训练模式革新**：实现无教练自主专业训练
- 数据分析深度**：从简单记录到智能诊断
- 用户体验提升**：从被动接受到主动交互

4. 遇到的重大问题及解决方案

问题类型	具体表现	解决方式	责任人
技术难点	用户上传高帧率视频时，前端页面卡顿，FPS从60骤降至15，动画撕裂、交互延迟	1. 与后端优化数据格式，改用二进制Buffer传输 2. 引入Web Worker离屏计算	杨宇琨（前端） 段子涛（后端）

		3. 迁移Three.js渲染至requestAnimationFrame循环外	
进度协调	LLM接口返回结构频繁变更，前端解析逻辑需反复重写	1. 使用Mock.js动态生成模拟数据 2. 制定版本化接口协议（v1/v2/v3） 3. 采用"分段验收"策略，优先确保关键路径可用	全体核心成员

5. 经验教训

5.1 成功经验

5.1.1 有效的团队管理方法（如每日站会/甘特图跟踪）

在项目中，我们采用了一系列有效的团队管理方法来确保项目的顺利进行。每日站会成为我们同步进度、解决障碍的关键环节。每次站会上，每个成员都会简短汇报昨天的工作进展、今天的计划以及遇到的任何问题。这种方法不仅提高了信息透明度，也促进了团队内部的知识共享和问题解决速度。例如，在一次站会上，段子涛提到前端渲染效果不稳定的问题，这一提示帮助杨宇琨提前半天发现了潜在的技术风险。

此外，我们还利用了甘特图对整个项目的时间线进行了详细的规划与跟踪。通过将大目标分解为小任务，并明确每个阶段的完成时间，使得原本模糊的“软件开发生命周期”概念变得更加具体和可操作。这有助于我们及时调整策略，确保各个模块按期交付。比如，当对接MediaPipe服务与大模型时，通过甘特图我们可以清晰地看到各阶段的任务分配及依赖关系，从而更有效地协调资源。

5.1.2 研发环节的高效执行策略

在处理点集数据转换为自然语言描述的过程中，王浩然采取了“先Mock数据再联调”的策略，大大提升了前后端协作效率。该策略允许前端开发人员基于模拟数据构建用户界面，同时后端根据前端反馈逐步完善接口定义，这种方式减少了不必要的等待时间，加快了整体开发节奏。

另一个成功的执行策略是在面对技术路径调整时采用的“双线并行、阶段整合”策略。当原定自研模型训练因数据量不足陷入困境时，团队迅速重组资源，一方面由段子涛和王浩然快速验证Mediapipe输出与大模型解析的兼容性，另一方面优化数据字段扩展性设计，为后续需求变更预留缓冲空间。这种灵活应对措施保证了项目能够在变化中找到最优解。

5.2 改进方向

5.2.1 时间规划不足导致的风险（如测试时间被压缩）

尽管我们在项目初期制定了详细的计划，但由于对某些任务复杂度估计不足，导致后期测试时间被严重压缩。特别是在对接MediaPipe服务与大模型的过程中，由于前期未搭建自动化测试流水线，最终不得不采用手动回归测试的方式弥补漏洞，极大地增加了工作量。未来需要更加精确地评估每个阶段所需的时间，并为不可预见的延迟留出足够的缓冲期。

5.2.2 技术预研不充分造成的问题

在项目早期，对于一些新技术的应用和技术细节的掌握不够深入，导致在实际开发过程中遇到了不少挑战。例如，在集成大语言模型时，我们低估了Prompt Engineering的复杂性，没有充分考虑到不同动作描述之间的上下文关联性和模型响应延迟的影响。因此，建议在未来的项目中加强对新技术的研究和实验，制定详细的技术可行性分析报告，以减少技术实现过程中的不确定性。同时，建立一套标准的反馈评估机制，以便更快地适应技术上的变化。

5.3 个人感悟——对于经验与教训

陈茁扬：

成功经验

回顾整个项目，最值得总结的成功经验是"分层治理"的管理理念在微服务架构中的实践。面对五个不同技术栈的微服务，我采用了"统一标准、分层管理"的策略：在基础设施层统一使用Nacos进行服务治理，在应用层制定统一的接口规范和错误码标准，在监控层通过ELK建立全链路日志追踪。这种方式既保证了系统的一致性，又给各个业务模块留出了技术选型的灵活空间。

特别值得一提的是"预防式监控"的理念实践。通过在关键业务节点埋点监控，我们能够在问题爆发前就发现系统的异常征兆。比如当MediaPipe服务的响应时间出现波动时，监控系统会自动告警，让我们能够提前进行负载调整，避免了多次潜在的系统崩溃。这种从"被动救火"到"主动预防"的运维思路转变，让整个团队对系统稳定性的信心大大增强。

改进方向

一个重要的改进方向是在团队协作工具的选择上。虽然我们使用了飞书文档进行协作，但在代码review和技术方案讨论上，缺乏更专业的工具支持。未来我会推动团队使用更系统化的协作平台，比如通过GitLab的MR机制进行代码审查，通过Confluence进行技术文档管理，让协作更加规范化和可追溯。

这次项目让我深刻理解了"架构师"这个角色的真正内涵：不仅要有扎实的技术功底，更要具备全局视野和协调能力。技术方案可以优化，但团队协作的经验和危机处理的能力，将是我职业生涯中最宝贵的财富。

杨宇琨：

成功经验

这次项目让我深刻体会到**结构化沟通**的价值。我们自发形成的"晨会15分钟+当日任务看板"机制，意外成为项目高效推进的关键——比如在MediaPipe坐标转换问题爆发时，正是站会上子涛随口提到的"Z轴缩放异常"提醒，让我们提前半天发现了潜在风险。而当我将大模型接口的联调任务拆解为"数据模拟→基础联调→异常处理"三个阶段，并配合使用甘特图跟踪时，原本混乱的进度突然变得清晰可控，这种**模块化+可视化**的管理方式，我会延续到未来的所有项目中。

改进方向

最大的教训来自对**技术债务**的低估。初期为赶进度，我们直接在前端硬编码了关键点索引，导致后期MediaPipe模型升级时，不得不耗费两天批量修改多处渲染逻辑。更痛心的是测试阶段——因前期未搭建自动化测试流水线，最终被迫用通宵手动回归测试，这种"省小时间赔大时间"的决策让我意识到：**每一个临时方案都应该标注技术债务标签**。下次我会在项目启动时，就强制预留15%的缓冲期专门处理这类"隐形债务"，毕竟，好的工程实践不是奢侈品，而是必需品。

段子涛：

成功经验

在项目推进过程中，最值得复盘的经验源于技术路径调整时的集体决策机制。当原定自研模型因数据量不足陷入瓶颈时，团队果断重构任务优先级：一边由我和王浩然快速验证Mediapipe输出与大模型解析的兼容性，另一边同步优化数据字段的扩展性设计，为后续需求变更预留缓冲空间。这种“分阶段验证、动态调整”的策略，源于每日站会对风险点的即时暴露与集体讨论——比如发现用户照片背景复杂影响检测精度时，团队立即分配资源进行图像预处理方案的头脑风暴，而非固守原计划。我逐渐明白，真正的工程能力不仅体现在代码质量上，更在于能否通过协作将碎片化技术整合为可交付的价值。这种认知转变，或许比任何具体技术都更具迁移价值。

改进方向

回望整个开发周期，最值得反思的是对需求模糊性的容忍度与测试环节的轻视。前期过度依赖“先跑起来再优化”的思路，导致关键点坐标定义偏差引发前端渲染卡顿，暴露出需求边界未清晰锁定的风险——若能在项目启动时强制制定“核心指标验收清单”，或许能大幅减少后期的接口重构成本。此外，技术债务的积累速度远超预期：为赶进度在代码中硬编码的参数配置，最终耗费数日批量修正；测试环节因前期未搭建自动化流水线，被迫用通宵手动回归测试弥补漏洞。这些教训印证了工程实践中“隐形债务”的破坏力。未来我会在项目规划阶段强制预留一定时间的缓冲期专门处理此类问题，并通过“分段验收+Mock数据先行”的策略，让测试流程与开发节奏深度绑定。真正的工程化思维，不只追求功能的快速落地，更要为系统的可持续演进留足弹性空间。

王浩然：

成功经验

回顾整个项目，最让我印象深刻的成功经验是跨模块协作中的“共情式沟通”与“数据先行”的策略。在处理点集数据与大模型API对接的过程中，我意识到单靠技术实现远远不够。为了让前端、后端和算法团队都能准确理解姿态评估逻辑的输出形式，我和段子涛一起设计了一套“通用样例库”，将关键点数据以自然语言+结构化JSON两种方式呈现，并附上评分说明和改进建议模板。这种方式不仅提升了各模块对输出结果的一致性理解，也大幅减少了因语义差异导致的反复调试问题。此外，在与杨宇琨合作优化用户报告展示时，我们采用了“先Mock数据再联调”的策略：即在接口尚未完成前，先用模拟数据构建前端展示框架，同时反向推动后端接口定义的完善。这种“以终为始”的开发思路，让前后端协作更加顺畅，也为后续类似项目的开发流程提供了可复用的经验。

改进方向

这次项目也暴露出我们在时间规划和技术预研方面的不足。首先是对任务依赖链的评估不够充分。在早期阶段，我以为只要把关键点转换成Prompt就可以直接调用大模型，但忽略了不同动作描述之间的上下文关联性和模型响应延迟的影响。由于前期没有预留足够的时间进行Prompt工程实验，导致中期不得不临时调整生成逻辑，影响了整体进度。如果当时能更早地介入大模型行为模拟，并建立一套标准的反馈评估机制，可能会减少很多试错成本。其次是在测试环节的准备不足。虽然我们在后期补充了大量手动测试用例，但由于缺乏自动化测试工具链的支持，导致每次模型输出格式变更都需要重新验证所有交互场景，效率很低。这让我意识到，技术实现只是第一步，如何通过测试保障系统的稳定性和可维护性，同样重要。未来，我会更加注重项目初期的“技术可行性验证”和“多维度测试计划制定”，特别是在涉及AI模型集成的项目中，提前设计好数据样本集、响应评估指标以及异常兜底策略，避免因模型不可控因素拖累整体进度。同时，也会在团队内部推动更多标准化的协作流程，比如统一的数据规范、接口Mock机制和自动化回归测试体系，让协作更高效、交付更可靠。

刘佳朋：

成功经验

项目成功的最大亮点在于团队成员间的高效协作与专业互补。我们团队由软件工程、赛艇运动员等不同背景的成员组成，通过定期的技术研讨会和需求对接会议，实现了理论与实践的深度融合。体育专业的同学提供了专业的划船技术指导和动作标准，帮助我们理解划船运动的生物力学原理；而软件工程专业的同学则专注于模型架构设计和系统实现。这种分工合作模式不仅提高了工作效率，更重要的是确保了技术方案的科学性和实用性。

改进方向

当前模型主要依靠坐标数据进行分析，未来应考虑融合更多模态的信息，如肌电信号、心率数据、视频图像等，构建更全面的运动员状态评估体系。同时，现有模型的推理速度还需进一步优化，以实现真正的实时分析和反馈。为了提升用户体验，我们需要优化建议的表达方式，使其更加直观易懂。计划开发可视化界面，通过3D动画展示标准动作与实际动作的对比，让运动员能够更直观地理解改进要点。

6. 致谢

6.1 指导老师的建议

在整个项目过程中，老师的建议起到了至关重要的作用。

其中一条尤为关键的指导是在我们面对数据量不足导致自研模型训练陷入瓶颈时提出的。导师建议我们不要仅仅局限于增加数据量这一条路径，而是可以考虑转向利用现有的大语言模型进行适配。这不仅帮助我们打破了项目的僵局，还引导团队探索了新的技术方向——将MediaPipe输出的结果直接转化为大模型可解析的形式，并通过场景化数据模拟提前验证交互流程的有效性。这种“双线并行、阶段整合”的策略最终证明是非常有效的，既发挥了成员在算法优化与工程落地上的专长，也加速了项目的进展。

6.2 组员间的互相支持

在这段充满挑战的开发旅程中，组员们之间的相互支持是项目成功不可或缺的一部分。例如，在遇到前端页面因用户上传高帧率视频而出现卡顿的问题时，原本负责后端的段子涛主动加入了问题解决小组。他与前端负责人杨宇琨紧密合作，共同探讨解决方案，从优化前后端数据格式到引入Web Worker离屏计算，再到迁移Three.js渲染至requestAnimationFrame循环外，最终解决了性能瓶颈，显著提升了用户体验。

另一个例子是在准备最终演示文档和展示材料的过程中，陈茁扬主动承担起了整理架构设计文档的任务，确保所有微服务之间的交互图清晰明了，同时协助其他成员完善各自的演示内容。他还组织了多次预演，提供宝贵的反馈意见，帮助整个团队更好地展现我们的成果。

这些实例只是众多互助行为中的冰山一角，它们充分体现了团队成员间无私的支持和协作精神，正是这种精神使得我们能够克服重重困难，最终完成了一个高质量且富有创新性的智能赛艇训练管理系统。